

CURSO PROFESIONAL · PYTHON

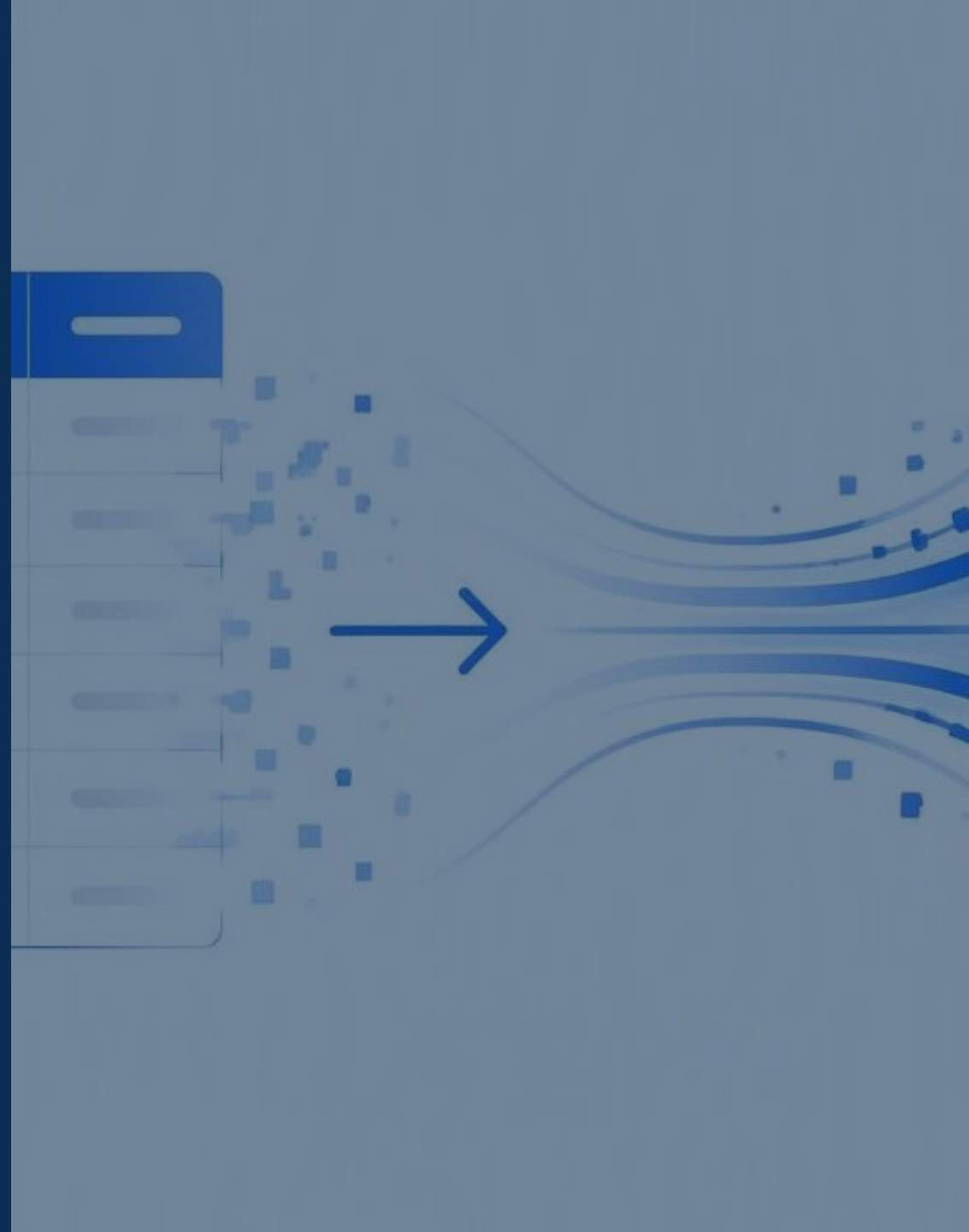
Pandas

Manipulación de datos en Python

De principiante a experto

10 módulos · Fundamentos → Nivel experto

Estudiantes y profesionales de Ciencia de Datos, Análisis, Ingeniería de Datos y desarrollo Python



Agenda del curso

01

Introducción

Qué es Pandas y por qué es el estándar

02

Fundamentos

Series, DataFrame, índices y tipos

03

Carga y exportación

CSV, Excel, JSON, APIs y SQL

04

Selección y filtrado

loc/iloc, consultas y ordenamiento

05

Limpieza de datos

Nulos, duplicados, fechas y outliers

06

Transformación

GroupBy, merge, pivot y ventanas

07

Visualización y estadística

Matplotlib, Seaborn y EDA

08

Rendimiento

Memoria, categoricals y vectorización

09

Casos reales

Ventas, RR.HH., finanzas, IoT y ML

10

Nivel experto

MultiIndex, Time Series, Spark y Fabric

MÓDULO 1

Introducción

Qué es Pandas y por qué se convirtió en el estándar de facto para manipular datos en Python.

¿Qué es Pandas?

Pandas es la librería de código abierto para análisis y manipulación de datos tabulares en Python. Ofrece estructuras rápidas y expresivas construidas sobre

NumPy.

In [1]:

```
import pandas as pd

# Crear un DataFrame a partir de un diccionario
df = pd.DataFrame({
    "producto": ["A", "B", "C"],
    "ventas":   [1200, 950, 1800],
})
print(df.describe())
```

Out [1]:

	ventas
count	3.00
mean	1316.67
max	1800.00

Ideas clave

- **Estructuras Series y DataFrame**
- Datos etiquetados por filas y columnas
- **Alineación automática por índice**
- Manejo nativo de valores faltantes
- **Base del stack de Data Science**
- Interopera con NumPy, Scikit-learn y más

Historia y evolución

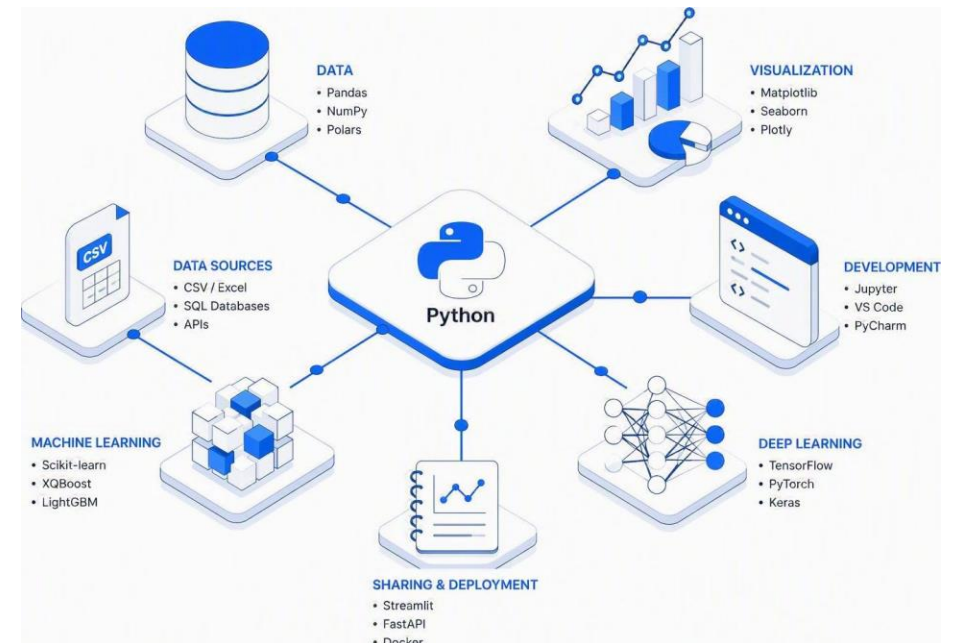
Creada por Wes McKinney; su nombre proviene de «panel data». Bajo el paraguas de NumFOCUS es hoy uno de los proyectos más usados del ecosistema Python.



Ecosistema Python para Data Science

Pandas es el punto de encuentro del flujo de datos: casi todas las librerías del ecosistema hablan «DataFrame».

- **Cómputo numérico**
- NumPy aporta los arrays de bajo nivel
- **Visualización**
- Matplotlib, Seaborn y Plotly leen DataFrames
- **Machine Learning**
- Scikit-learn, XGBoost y LightGBM
- **Escala y despliegue**
- Spark, Streamlit, FastAPI y Docker



Pandas vs Excel

Ambos manejan tablas, pero escalan y se automatizan de forma muy distinta.

Excel

- Ideal para conjuntos pequeños (< 100 K filas)
- Interfaz visual e inmediata para no técnicos
- Automatización limitada y manual (VBA)
- Difícil de versionar y reproducir
- Se degrada con grandes volúmenes

Pandas

- Procesa millones de filas (límite = RAM)
- Flujos reproducibles y versionables en código
- Automatización total y programable
- Integra ML, APIs y web scraping
- Curva de aprendizaje más pronunciada

Pandas vs SQL

No compiten: se complementan. Muchos flujos profesionales usan ambos.

SQL

- Escala a miles de millones de filas
- Declarativo y optimizado por el motor
- Gobernanza y datos centralizados
- Débil en transformaciones complejas y stats
- Sin capacidades nativas de ML

Pandas

- Limitado por la memoria disponible
- Imperativo y flexible en Python
- Ideal para feature engineering y modelado
- Estadística, series temporales y gráficos
- Puente natural hacia Scikit-learn

Casos de uso empresariales

Pandas está presente en prácticamente todas las áreas que trabajan con datos.

Ventas

Reporting, cohortes y
forecasting de demanda

Finanzas

Conciliación, riesgo y series
temporales

RR.HH.

Rotación, headcount y
análisis de nómina

IoT · ML

Sensores, resampling y
feature engineering

MÓDULO 2

Fundamentos

Series, DataFrame, índices, columnas y tipos de datos: los cimientos de todo análisis.

Instalación

Instala Pandas con pip o conda. Se recomienda un entorno virtual y añadir motores opcionales para Excel y Parquet.

In [1]:

```
# pip (entorno virtual recomendado)
pip install pandas

# Motores opcionales frecuentes
pip install pandas openpyxl pyarrow

# Con conda
conda install -c conda-forge pandas
```

Recomendaciones

- **Usa un entorno virtual**
 - venv, conda o uv para aislar dependencias
- **pyarrow**
 - Backend Arrow y lectura rápida de Parquet
- **openpyxl**
 - Lectura y escritura de archivos .xlsx
- **Fija versiones**
 - requirements.txt para reproducibilidad

Importación de la librería

La convención universal es importar Pandas como pd. Verifica la versión instalada para asegurar compatibilidad.

In [1]:

```
import pandas as pd
import numpy as np

print(pd.__version__)
print(pd.show_versions()) # entorno completo
```

Out [1]:

```
3.0.4
```

Convención estándar

- **import pandas as pd**
- El alias pd es universal en la comunidad
- **Acompáñalo de numpy as np**
- Ambos trabajan de la mano
- **pd.__version__**
- Comprueba la versión antes de depurar

Estructura: Series

Una Series es un array unidimensional etiquetado: valores + un índice. Es la columna elemental de Pandas.

In [1]:

```
s = pd.Series([10, 20, 30], index=["a", "b", "c"], name="ventas")

print(s["b"])           # acceso por etiqueta
print(s.mean())         # operaciones vectorizadas
print(s * 2)            # broadcasting
```

Out [1]:

```
20
20.0
a    20
b    40
c    60
```

Anatomía de una Series

- **Índice etiquetado**
- Acceso por posición o por etiqueta
- **Un único dtype**
- Homogénea y vectorizada
- **name**
- Se convierte en el nombre de columna

Estructura: DataFrame

El DataFrame es una tabla bidimensional: un conjunto de Series que comparten el mismo índice de filas.

In [1]:

```
df = pd.DataFrame({
    "empleado": ["Ana", "Luis", "Marta"],
    "area":     ["Ventas", "IT", "Ventas"],
    "salario":  [45000, 52000, 48000],
})
print(df.shape)      # (filas, columnas)
```

Out [1]:

```
(3, 3)
```

Características

- **Filas × columnas**
- Estructura tabular heterogénea
- **Columnas = Series**
- Cada columna tiene su propio dtype
- **Índice compartido**
- Alinea todas las columnas por fila
- **.shape / .info()**
- Dimensiones y metadatos al instante

Índices

El índice identifica cada fila y habilita la alineación automática. Puede ser numérico, de texto o temporal.

In [1]:

```
df = df.set_index("empleado") # columna -> índice
print(df.loc["Luis"])        # acceso por etiqueta

df = df.reset_index()         # volver a rango 0..n
df.index = df.index + 1       # reindexar manualmente
```

Por qué importa

- **Identidad de fila**
- Cada registro tiene su etiqueta
- **Alineación automática**
- Operaciones entre objetos por índice
- **set_index / reset_index**
- Promueve o degrada columnas a índice
- **Base del MultiIndex**
- Índices jerárquicos avanzados

Columnas

Las columnas se crean, renombran y eliminan con operaciones vectorizadas, sin bucles.

In [1]:

```
df["bono"] = df["salario"] * 0.10      # nueva columna
df = df.rename(columns={"area": "departamento"})
df = df.drop(columns=["bono"])          # eliminar
df = df.assign(salario_k=df["salario"] / 1000)
```

Operaciones habituales

- **Crear**
 - Asignación directa o .assign()
- **Renombrar**
 - .rename(columns=...)
- **Eliminar**
 - .drop(columns=[...])
- **Vectorizado**
 - Opera sobre toda la columna a la vez

Tipos de datos (dtypes)

Elegir el dtype correcto mejora memoria y rendimiento. Pandas 2.x añade tipos nullable y respaldados por Arrow.

dtype	Descripción	Ejemplo	Uso recomendado
int64 / Int64	Enteros (Int64 admite nulos)	1, 42, -7	Conteos, identificadores
float64	Números decimales	3.14, NaN	Métricas continuas
bool	Booleanos	True / False	Máscaras y flags
object / string	Texto (string es más eficiente)	"Ana"	Nombres, categorías libres
category	Texto de baja cardinalidad	"Ventas"	Ahorra memoria (+80%)
datetime64[ns]	Fechas y horas	2026-07-04	Series temporales

Inspección de datos

Antes de analizar, inspecciona. Estos métodos dan una radiografía inmediata del DataFrame.

In [1]:

```
df.head()          # primeras filas
df.info()          # dtypes, no-nulos y memoria
df.describe()      # estadística de columnas numéricas
df.shape           # (filas, columnas)
df.dtypes          # tipo por columna
df.isna().sum()    # nulos por columna
```

Rutina de arranque

- **head() / tail()**
 - Vistazo rápido a los datos
- **info()**
 - Tipos, nulos y consumo de memoria
- **describe()**
 - Resumen estadístico
- **isna().sum()**
 - Diagnóstico de valores faltantes

MÓDULO 3

Carga y exportación de datos

Leer y escribir desde CSV, Excel, JSON, APIs y bases de datos SQL con buenas prácticas.

Lectura de CSV

`read_csv` es la función de importación más usada. Sus parámetros controlan separadores, tipos, fechas y memoria.

In [1]:

```
df = pd.read_csv(
    "ventas.csv",
    sep=",", encoding="utf-8",
    parse_dates=["fecha"],
    dtype={"codigo": "string"},
    usecols=["fecha", "codigo", "importe"],
)
```

Parámetros clave

- **parse_dates**
 - Convierte columnas a datetime al leer
- **dtype**
 - Fija tipos y evita inferencias erróneas
- **usecols**
 - Lee solo lo necesario: menos memoria
- **chunksize**
 - Procesa archivos enormes por lotes

Lectura de Excel

`read_excel` importa hojas de cálculo. Requiere el motor `openpyxl` para archivos `.xlsx`.

In [1]:

```
df = pd.read_excel(
    "reporte.xlsx",
    sheet_name="Q1",          # nombre o índice de hoja
    header=0,
    engine="openpyxl",
)
# Leer todas las hojas en un diccionario
hojas = pd.read_excel("reporte.xlsx", sheet_name=None)
```

A tener en cuenta

- **sheet_name**
 - Una hoja, varias o None (todas)
- **engine=openpyxl**
 - Necesario para `.xlsx`
- **header / skiprows**
 - Salta títulos y celdas combinadas

Lectura de JSON

`read_json` y `json_normalize` convierten estructuras JSON, incluso anidadas, en tablas planas.

In [1]:

```
df = pd.read_json("datos.json")

# JSON anidado -> tabla plana
import json
data = json.load(open("api.json"))
df = pd.json_normalize(data, record_path="items",
                       meta=["cliente", "fecha"])
```

Claves

- **read_json**
 - JSON tabular directo a DataFrame
- **json_normalize**
 - Aplana estructuras anidadas
- **record_path / meta**
 - Selecciona listas y campos padre

Lectura desde APIs

Combina requests para obtener datos de una API REST y Pandas para estructurarlos.

In [1]:

```
import requests

resp = requests.get("https://api.ejemplo.com/ventas",
                    params={"anio": 2026}, timeout=30)
resp.raise_for_status()
df = pd.json_normalize(resp.json()["data"])
print(df.head())
```

Buenas prácticas

- **raise_for_status()**
- Falla rápido ante errores HTTP
- **timeout**
- Evita bloqueos indefinidos
- **Paginación**
- Itera y concatena páginas
- **json_normalize**
- Aplana la respuesta a tabla

Lectura desde bases de datos SQL

`read_sql` ejecuta consultas contra cualquier base soportada por SQLAlchemy y devuelve un DataFrame.

In [1]:

```
from sqlalchemy import create_engine

engine = create_engine("postgresql://user:pwd@host/db")
df = pd.read_sql(
    "SELECT fecha, importe FROM ventas WHERE anio = 2026",
    engine, parse_dates=["fecha"],
)
```

Recomendaciones

- **SQLAlchemy engine**
- Conexión reutilizable y segura
- **Filtra en la consulta**
- Trae solo lo necesario
- **params**
- Consultas parametrizadas, sin inyección
- **chunksize**
- Resultados grandes por lotes

Guardado de resultados

Exporta a múltiples formatos. Parquet es la mejor opción para datos analíticos por tamaño y velocidad.

In [1]:

```
df.to_csv("salida.csv", index=False)
df.to_excel("salida.xlsx", sheet_name="Resumen", index=False)
df.to_parquet("salida.parquet")          # columnar + comprimido
df.to_json("salida.json", orient="records")
```

Elegir formato

- **Parquet**
- Columnar, comprimido y tipado: ideal
- **CSV**
- Universal pero sin tipos ni compresión
- **index=False**
- Evita una columna índice extra

Formatos soportados

Pandas lee y escribe una amplia variedad de formatos con la familia `read_*` / `to_*`.

Formato	Lectura	Escritura	Cuándo usarlo
CSV	<code>read_csv</code>	<code>to_csv</code>	Intercambio universal
Excel	<code>read_excel</code>	<code>to_excel</code>	Informes para negocio
Parquet	<code>read_parquet</code>	<code>to_parquet</code>	Analítica y data lakes
JSON	<code>read_json</code>	<code>to_json</code>	APIs y datos anidados
SQL	<code>read_sql</code>	<code>to_sql</code>	Bases de datos relacionales
Feather	<code>read_feather</code>	<code>to_feather</code>	Intercambio rápido con Arrow

Buenas prácticas de importación

Una importación cuidada evita el 80 % de los problemas posteriores de calidad y rendimiento.

- **Especifica dtypes al leer**
- Evita inferencias erróneas y ahorra memoria
- **Prefiere Parquet sobre CSV**
- Más rápido, tipado y comprimido
- **Filtra en origen**
- usecols, consultas SQL, columnas de Parquet
- **Procesa por lotes lo enorme**
- chunksize cuando no cabe en RAM
- **Reduce memoria tras cargar**
- Downcasting y categoricals inmediatos
- **Valida el esquema**
- Comprueba columnas, tipos y nulos al inicio

MÓDULO 4

Selección y filtrado

Elegir columnas y filas con precisión: loc, iloc, condiciones lógicas y consultas complejas.

Selección de columnas

Selecciona una o varias columnas con corchetes. Una lista devuelve un DataFrame; un nombre suelto, una Series.

In [1]:

```
df["salario"]           # Series (una columna)
df[["empleado", "salario"]] # DataFrame (varias)

# Selección por tipo
df.select_dtypes(include="number")
df.filter(like="fecha")      # por patrón de nombre
```

Recuerda

- **df['col']**
- Devuelve una Series
- **df[['a','b']]**
- Devuelve un DataFrame
- **select_dtypes**
- Filtra columnas por tipo
- **filter(like=...)**
- Selecciona por patrón de nombre

loc e iloc

loc selecciona por etiqueta; iloc por posición entera. Son la forma canónica y segura de indexar.

In [1]:

```
df.loc[3, "salario"]          # etiqueta de fila y columna
df.loc[df["area"] == "IT", ["empleado", "salario"]]

df.iloc[0]                   # primera fila por posición
df.iloc[0:5, 0:3]            # rango por posición
```

loc vs iloc

- **loc = etiquetas**
- Incluye el extremo final del rango
- **iloc = posiciones**
- Excluye el final, como en Python
- **Asignación segura**
- `df.loc[máscara, col] = valor`
- **Evita el encadenado**
- Previene SettingWithCopyWarning

Filtrado condicional

Una condición booleana genera una máscara que selecciona las filas que cumplen el criterio.

In [1]:

```
mask = df["salario"] > 48000
df[mask]

df[df["area"].isin(["Ventas", "IT"])]
df[df["empleado"].str.startswith("A")]
```

Out [1]:

	empleado	area	salario
1	Luis	IT	52000

Herramientas de máscara

- **Comparaciones**
 - >, <, ==, != vectorizadas
- **isin()**
 - Pertenencia a un conjunto
- **str.***
 - Filtrado sobre texto
- **between()**
 - Rangos de valores

Operadores lógicos

Combina condiciones con & (y), | (o) y ~ (no). Cada condición debe ir entre paréntesis.

In [1]:

```
# Paréntesis obligatorios por precedencia
df[(df["area"] == "Ventas") & (df["salario"] > 45000)]

df[(df["area"] == "IT") | (df["salario"] > 50000)]

df[~(df["area"] == "Ventas")]      # negación
```

Reglas de oro

- & | ~
- No uses and / or / not de Python
- **Paréntesis**
 - Envuelve cada condición
- **Vectorizado**
 - Opera sobre toda la columna

Consultas complejas

`query()` expresa filtros como texto legible; `eval()` evalúa expresiones de forma eficiente en grandes tablas.

In [1]:

```
df.query("area == 'Ventas' and salario > 45000")

umbral = 50000
df.query("salario > @umbral")      # variable con @

df.eval("bono = salario * 0.1", inplace=True)
```

Ventajas

- **Legibilidad**
 - Filtros como frases, sin ruido de sintaxis
- **@variable**
 - Referencia variables del entorno
- **Rendimiento**
 - eval acelera expresiones en tablas grandes

Ordenamiento

Ordena por valores o por índice, con múltiples claves y direcciones, para ranking y presentación.

In [1]:

```
df.sort_values("salario", ascending=False)

df.sort_values(["area", "salario"],
               ascending=[True, False])

df.sort_index()           # ordenar por índice
df.nlargest(3, "salario") # top-N directo
```

Opciones

- **sort_values**
 - Una o varias claves, ascendente/descendente
- **sort_index**
 - Ordena por la etiqueta de fila
- **nlargest / nsmallest**
 - Top-N sin ordenar todo

Indexación avanzada

Combina condiciones, selección de columnas y asignación en una sola instrucción con loc.

In [1]:

```
# Asignación condicional segura
df.loc[df["salario"] > 50000, "categoria"] = "Senior"

# where mantiene forma y sustituye lo que no cumple
df["salario"].where(df["salario"] > 0, other=0)

# Selección booleana + columnas concretas
df.loc[df["area"] == "IT", ["empleado", "salario"]]
```

Patrones potentes

- **loc[máscara, col] = x**
- Actualiza subconjuntos sin warnings
- **where / mask**
- Reemplazo condicional conservando forma
- **Combina filas y columnas**
- Todo en una expresión

Ejercicios prácticos

Practica con un DataFrame de empleados (empleado, area, salario, antigüedad). Intenta resolverlos sin bucles.

- **Desafío 1 · Filtrado**
 - Empleados de Ventas con salario > 45 000
- **Desafío 2 · Multi-condición**
 - Área IT O antigüedad mayor a 5 años
- **Desafío 3 · Ranking**
 - Top 3 salarios con nlargest
- **Desafío 4 · Asignación**
 - Etiqueta 'Senior' si salario > 50 000
- **Desafío 5 · query**
 - Reescribe el desafío 1 usando query()

MÓDULO 5

Limpieza de datos

Nulos, duplicados, tipos, fechas y outliers: convertir datos crudos en datos confiables.



Valores nulos

Detecta, cuenta y trata los valores faltantes. Elige entre eliminar o imputar según el contexto del negocio.

In [1]:

```
df.isna().sum()           # nulos por columna

df.dropna(subset=["salario"]) # elimina filas sin salario
df["salario"].fillna(df["salario"].median()) # imputa
df.ffill()                 # arrastra el último válido
```

Estrategias

- **Detectar**
 - isna().sum() y heatmaps de nulos
- **Eliminar**
 - dropna cuando sobran datos
- **Imputar**
 - media, mediana, moda o ffill/bfill
- **Decidir con criterio**
 - El método depende del negocio

Duplicados

Los registros repetidos distorsionan cualquier métrica. Identifícalos y elimínalos con control total.

In [1]:

```
df.duplicated().sum()          # cuántas filas repetidas

df = df.drop_duplicates()      # elimina duplicados exactos
df = df.drop_duplicates(
    subset=["email"], keep="last")  # por columnas clave
```

Control fino

- **duplicated()**
 - Marca las filas repetidas
- **subset**
 - Define qué columnas identifican un duplicado
- **keep**
 - first, last o False (elimina todas)

Conversión de tipos

Convertir al dtype correcto corrige errores y reduce memoria. `astype` y `to_numeric` son tus aliados.

In [1]:

```
df["codigo"] = df["codigo"].astype("string")
df["cantidad"] = pd.to_numeric(df["cantidad"],
                              errors="coerce") # inválidos -> NaN
df["activo"] = df["activo"].astype("bool")
df["area"] = df["area"].astype("category")
```

Métodos

- **astype**
- Conversión explícita de tipo
- **to_numeric / to_datetime**
- Con `errors='coerce'` para robustez
- **category**
- Menos memoria en texto repetido

Manejo de fechas

`to_datetime` interpreta cadenas como fechas y desbloquea el accesor `.dt` para extraer componentes.

In [1]:

```
df["fecha"] = pd.to_datetime(df["fecha"],
                             format="%Y-%m-%d")
df["anio"] = df["fecha"].dt.year
df["mes"] = df["fecha"].dt.month_name()
df["dia_semana"] = df["fecha"].dt.day_name()
```

El accesor `.dt`

- **to_datetime**
- Convierte texto a `datetime64`
- **.dt.year / month / day**
- Extrae componentes temporales
- **format**
- Acelera y evita ambigüedades

Tratamiento de outliers

Los valores atípicos sesgan medias y modelos. El rango intercuartílico (IQR) es un método robusto para acotarlos.

In [1]:

```
q1, q3 = df["importe"].quantile([0.25, 0.75])
iqr = q3 - q1
low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr

df["importe"] = df["importe"].clip(low, high) # winsorize
outliers = df[(df["importe"] < low) | (df["importe"] > high)]
```

Enfoques

- **Regla del IQR**
- $1.5 \times$ rango intercuartílico
- **clip()**
- Acota (winsoriza) los extremos
- **Z-score**
- Alternativa para datos normales
- **Investiga antes**
- ¿Error o dato real y valioso?

Estandarización de datos

Homogeneiza texto y escalas para que los datos sean comparables y consistentes.

In [1]:

```
df["area"] = df["area"].str.strip().str.lower()
df["area"] = df["area"].replace({"ventas ": "ventas"})

# Escalado min-max de una métrica numérica
col = df["salario"]
df["salario_norm"] = (col - col.min()) / (col.max() - col.min())
```

Qué estandarizar

- **Texto**
 - strip, lower y mapeos de categorías
- **Escalas**
 - Normalización min-max o z-score
- **Formatos**
 - Fechas, monedas y unidades uniformes

Calidad de datos

La calidad se mide en dimensiones. Un checklist evita sorpresas antes de analizar o modelar.

- **Compleitud**
 - ¿Cuántos nulos y por qué?
- **Unicidad**
 - Sin duplicados en las claves
- **Validez**
 - Tipos y rangos correctos
- **Consistencia**
 - Categorías y unidades homogéneas
- **Exactitud**
 - Coincide con la fuente de verdad
- **Actualidad**
 - Datos vigentes para la decisión

Caso práctico de limpieza

Un pipeline de limpieza encadena pasos legibles. El método chaining lo hace claro y reproducible.

In [1]:

```
limpio = (  
    pd.read_csv("crudo.csv")  
    .drop_duplicates()  
    .assign(fecha=lambda d: pd.to_datetime(d["fecha"]))  
    .dropna(subset=["cliente"])  
    .astype({"region": "category"})  
)  
print(limpio.shape)
```

Out [1]:

```
(12480, 8)
```

Method chaining

- **Flujo lineal**
- Cada paso recibe el resultado anterior
- **Sin variables intermedias**
- Menos ruido, más claridad
- **Reproducible**
- Fácil de auditar y versionar

MÓDULO 6

Transformación y análisis

GroupBy, agregaciones, merge/join, pivot y ventanas: de datos a información.

GroupBy: split-apply-combine

GroupBy divide los datos por clave, aplica una función a cada grupo y combina los resultados.



Agregaciones

agg permite calcular múltiples métricas por grupo y renombrar el resultado en una sola llamada.

In [1]:

```
df.groupby("area")["salario"].mean()

df.groupby("area").agg(
    salario_medio=("salario", "mean"),
    plantilla=("empleado", "count"),
    salario_max=("salario", "max"),
)
```

Out [1]:

area	salario_medio	plantilla
IT	52000.0	1
Ventas	46500.0	2

Named aggregation

- **agg(nombre=(col, func))**
- Varias métricas y nombres claros
- **Funciones múltiples**
- mean, sum, count, min, max, std
- **Personalizadas**
- Lambdas o funciones propias

Merge

merge combina DataFrames por columnas clave, al estilo de un JOIN de SQL.

In [1]:

```
ventas.merge(clientes, on="cliente_id", how="left")

ventas.merge(productos,
             left_on="prod", right_on="id",
             how="inner", suffixes=("_v", "_p"))
```

Tipos de join

- **how='inner'**
 - Solo coincidencias en ambos
- **how='left'**
 - Conserva todo el izquierdo
- **on / left_on / right_on**
 - Claves iguales o distintas
- **validate**
 - Comprueba cardinalidad 1:1, 1:m

Join

join combina por índice de forma concisa. Es cómodo cuando los DataFrames ya comparten índice.

In [1]:

```
clientes = clientes.set_index("cliente_id")
pedidos = pedidos.set_index("cliente_id")

pedidos.join(clientes, how="left")    # une por índice
# equivalente a un merge sobre el índice
```

join vs merge

- **join = por índice**
- Sintaxis breve cuando el índice es la clave
- **merge = por columnas**
- Más flexible y explícito
- **Mismo motor**
- join se apoya en merge internamente

Concatenación

concat apila DataFrames vertical u horizontalmente. Ideal para unir periodos o fuentes homogéneas.

In [1]:

```
pd.concat([enero, febrero, marzo], axis=0,  
          ignore_index=True)          # apila filas  
  
pd.concat([df_a, df_b], axis=1)        # une columnas  
  
pd.concat([q1, q2], keys=["Q1", "Q2"]) # índice jerárquico
```

Ejes y claves

- **axis=0**
- Apila filas (mismos campos)
- **axis=1**
- Añade columnas (mismo índice)
- **keys**
- Etiqueta el origen con MultiIndex

Pivot Tables

`pivot_table` resume datos en formato matriz, como las tablas dinámicas de Excel pero programables.

In [1]:

```
df.pivot_table(  
    index="area",  
    columns="anio",  
    values="salario",  
    aggfunc="mean",  
    margins=True,           # totales por fila y columna  
)
```

Anatomía

- **index / columns**
 - Dimensiones de la matriz
- **values + aggfunc**
 - Qué se resume y cómo
- **margins**
 - Totales automáticos
- **fill_value**
 - Rellena huecos de la tabla

Crosstab

crosstab construye tablas de frecuencia y contingencia entre dos o más variables categóricas.

In [1]:

```
pd.crosstab(df["area"], df["nivel"])

pd.crosstab(df["area"], df["nivel"],
            normalize="index")      # proporciones por fila

pd.crosstab(df["area"], df["nivel"],
            values=df["salario"], aggfunc="mean")
```

Usos

- **Frecuencias**
 - Conteo cruzado de categorías
- **normalize**
 - Proporciones por fila, columna o total
- **values + aggfunc**
 - Cruces con métricas agregadas

Ventanas analíticas

Las funciones de ventana calculan métricas móviles y acumuladas sin colapsar las filas.

In [1]:

```
df["media_7d"] = df["ventas"].rolling(window=7).mean()
df["acumulado"] = df["ventas"].cumsum()
df["var_pct"] = df["ventas"].pct_change()

df["media_exp"] = df["ventas"].ewm(span=7).mean() # ponderada
```

Familia de ventanas

- **rolling**
 - Ventana móvil (media, suma, std)
- **expanding**
 - Ventana acumulada creciente
- **ewm**
 - Ponderación exponencial
- **cumsum / pct_change**
 - Acumulados y variaciones

MÓDULO 7

Visualización y estadística

De los números al insight: Matplotlib, Seaborn, estadística descriptiva y EDA.

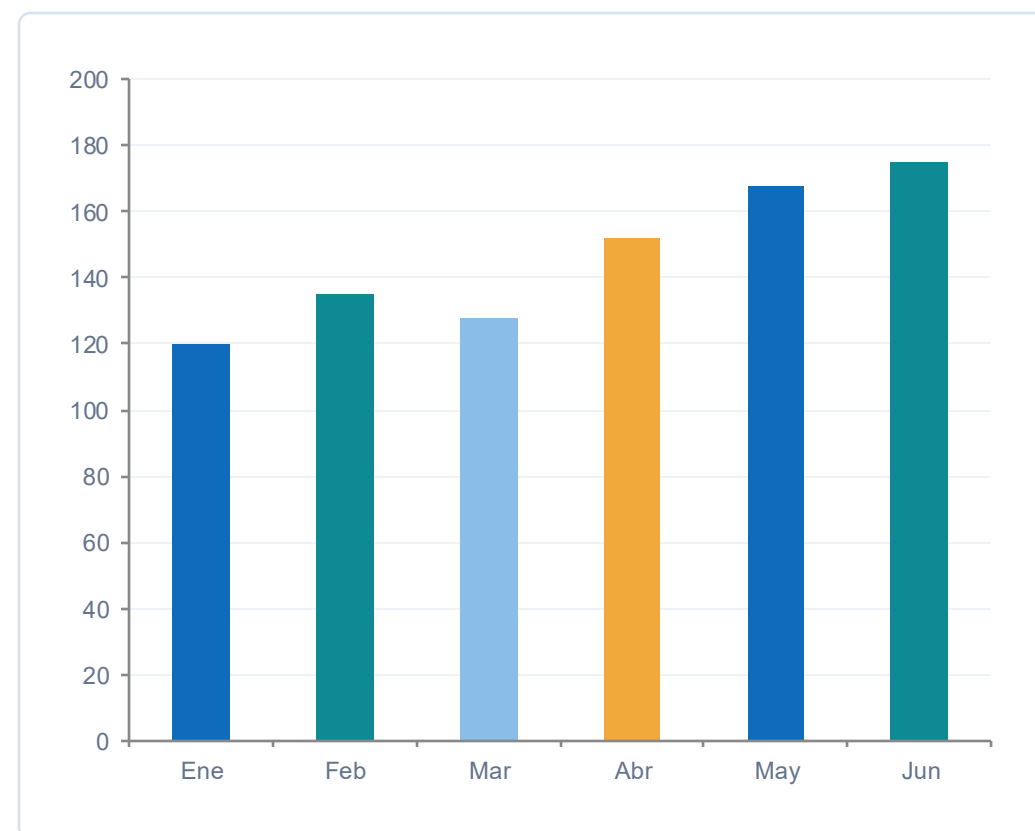
Integración con Matplotlib

Pandas incluye `.plot()`, un envoltorio de Matplotlib para graficar directamente desde el DataFrame.

In [1]:

```
import matplotlib.pyplot as plt

(df.groupby("mes")["ventas"].sum()
 .plot(kind="bar"))
plt.title("Ventas por mes")
plt.tight_layout()
```



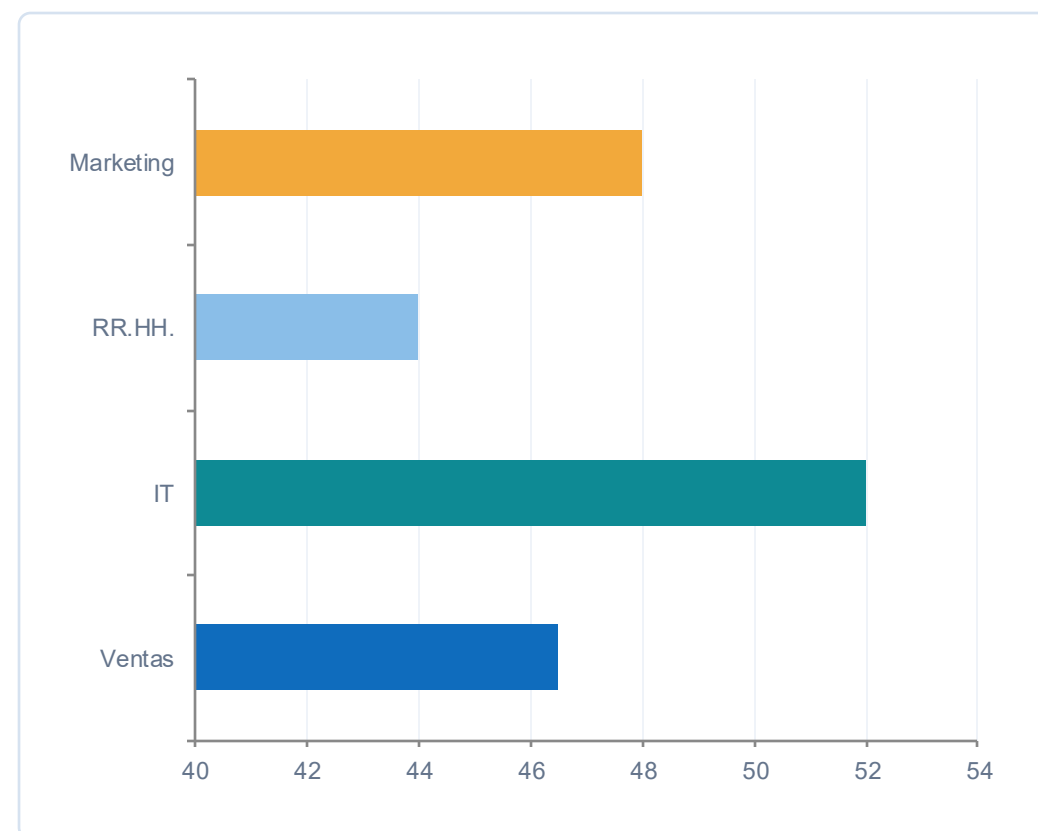
Integración con Seaborn

Seaborn produce gráficos estadísticos elegantes que consumen DataFrames de forma nativa.

In [1]:

```
import seaborn as sns

sns.boxplot(data=df, x="area", y="salario")
sns.heatmap(df.corr(numeric_only=True),
            annot=True, cmap="Blues")
```



Estadística descriptiva

describe() resume la distribución de cada columna numérica; también hay métodos individuales.

Método	Qué calcula	Método	Qué calcula
mean()	Media aritmética	median()	Mediana (robusta)
std()	Desviación estándar	var()	Varianza
min() / max()	Extremos	quantile()	Percentiles
describe()	Resumen completo	value_counts()	Frecuencias por valor
corr()	Matriz de correlación	mode()	Valor más frecuente

Correlaciones

`corr()` mide la relación lineal entre variables numéricas, base para selección de features y EDA.

In [1]:

```
corr = df.corr(numeric_only=True)      # Pearson por defecto
corr["ventas"].sort_values(ascending=False)

df.corr(method="spearman")             # relación monotónica
```

Out [1]:

```
ventas      1.00
marketing   0.82
precio     -0.41
```

Interpretación

- **Rango -1 a 1**
- Signo = dirección; magnitud = fuerza
- **Pearson vs Spearman**
- Lineal frente a monotónica
- **Correlación ≠ causalidad**
- No implica causa-efecto

Análisis exploratorio (EDA)

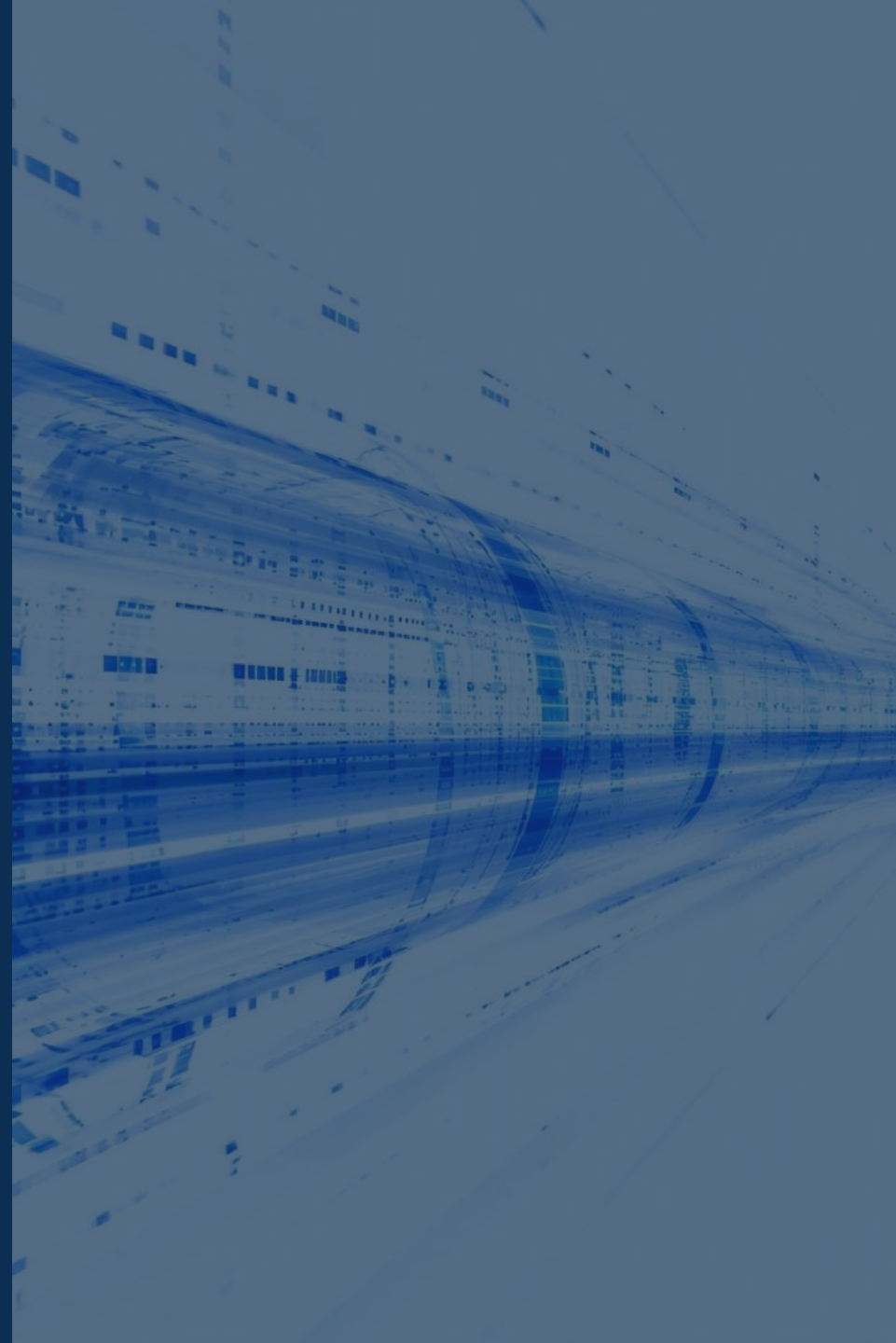
El EDA es el proceso sistemático de conocer los datos antes de modelar. Un flujo repetible da resultados fiables.



MÓDULO 8

Rendimiento y optimización

Menos memoria y más velocidad: dtypes, categoricals, vectorización y grandes volúmenes.



Optimización de memoria

Reducir el tipo de dato a la mínima precisión necesaria puede recortar la memoria entre un 50 % y un 90 %.

In [1]:

```
df.memory_usage(deep=True).sum() / 1e6    # MB reales

df["cantidad"] = pd.to_numeric(df["cantidad"],
                               downcast="integer")
df["precio"] = pd.to_numeric(df["precio"],
                             downcast="float")
```

Palancas de memoria

- **memory_usage(deep=True)**
 - Mide el consumo real
- **Downcasting**
 - int64→int8/16/32, float64→float32
- **Nullable / Arrow**
 - Tipos eficientes de Pandas 2.x
- **Regla práctica**
 - Pandas usa 2–5× el tamaño del archivo

Categoricals

Convertir columnas de texto de baja cardinalidad a category almacena códigos enteros en lugar de repetir cadenas.

+80%

Ahorro de memoria típico en texto repetido

<50%

Valores únicos: umbral recomendado

Veloz

GroupBy, sort y filtros más rápidos

Ejemplo: `df["area"] = df["area"].astype("category")`

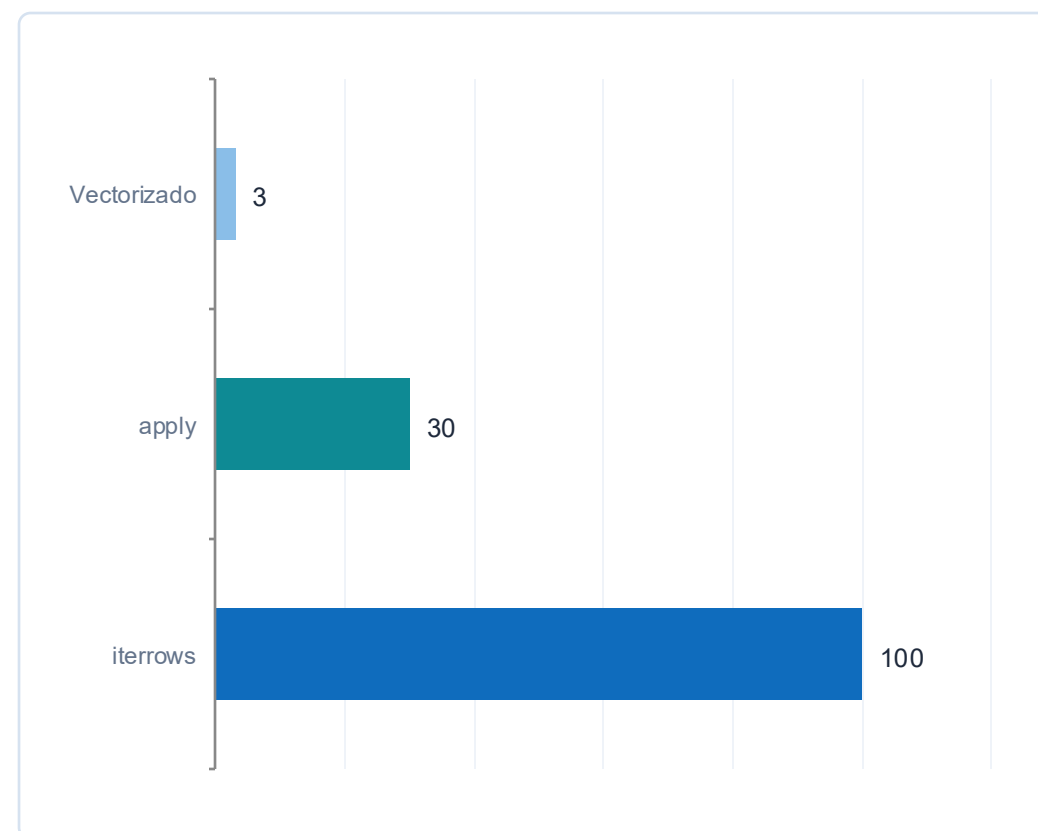
Vectorización

Operar sobre columnas enteras (vectorizado) es órdenes de magnitud más rápido que iterar fila a fila.

In [1]:

```
# LENTO: bucle explícito
for i, row in df.iterrows():
    df.at[i, "total"] = row["p"] * row["q"]

# RÁPIDO: vectorizado
df["total"] = df["p"] * df["q"]
```



Aplicación de funciones eficientes

Cuando no hay operación vectorizada directa, elige la herramienta más eficiente en lugar de un bucle.

In [1]:

```
df["ventas"].map({"A": 1, "B": 2})      # sustitución rápida
df["nombre"].str.upper()                 # accesor vectorizado
np.where(df["s"] > 0, "pos", "neg")      # condicional vectorizado

df["x"].apply(mi_funcion)                # último recurso
```

Orden de preferencia

- **1 · Vectorizado**
 - Operaciones y accesorios str/dt
- **2 · map / np.where**
 - Sustituciones y condicionales
- **3 · apply**
 - Flexible pero más lento
- **Evita iterrows**
 - Casi siempre la peor opción

Rendimiento en grandes volúmenes

Cuando los datos superan la RAM, hay estrategias para seguir trabajando con Pandas o dar el salto de escala.

- **Procesa por lotes**
 - `read_csv(chunksize=...)` y agrega por partes
- **Lee solo lo necesario**
 - `usecols` y columnas de Parquet
- **Tipos eficientes**
 - `category`, `downcasting` y backend Arrow
- **Formato columnar**
 - Parquet en vez de CSV
- **Escala horizontal**
 - Polars, DuckDB o Spark si no cabe
- **Perfila**
 - `%timeit` y `memory_usage` antes de optimizar

MÓDULO 9

Casos reales

Pandas aplicado a ventas, RR.HH., finanzas, marketing, IoT y machine learning.

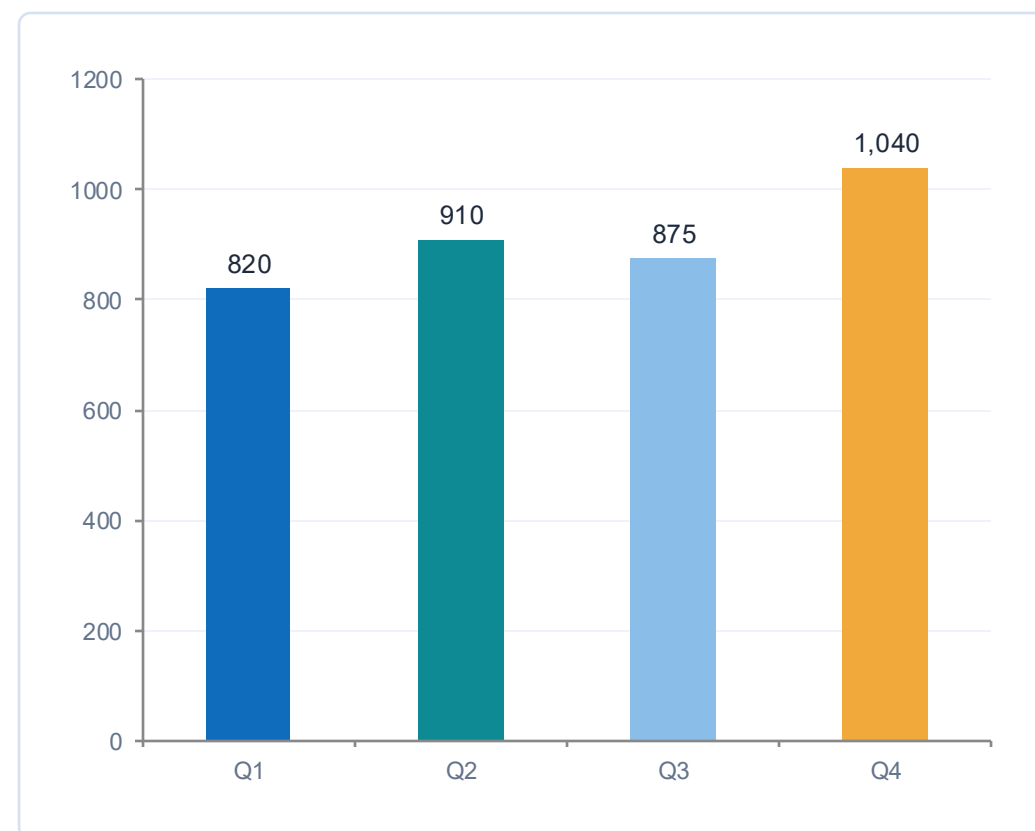


Análisis de ventas

Resume ventas por región y periodo para detectar tendencias y priorizar acciones comerciales.

In [1]:

```
ventas = pd.read_parquet("ventas.parquet")
resumen = (ventas
            .groupby(ventas["fecha"].dt.quarter)["importe"]
            .sum())
resumen.plot(kind="bar")
```



Recursos Humanos

Analiza rotación, headcount y equidad salarial para apoyar decisiones de personas basadas en datos.

In [1]:

```
# Tasa de rotación por departamento
rotacion = (bajas.groupby("area").size()
            / plantilla.groupby("area").size())

# Brecha salarial por género
df.groupby("genero")["salario"].median()
```

Métricas de personas

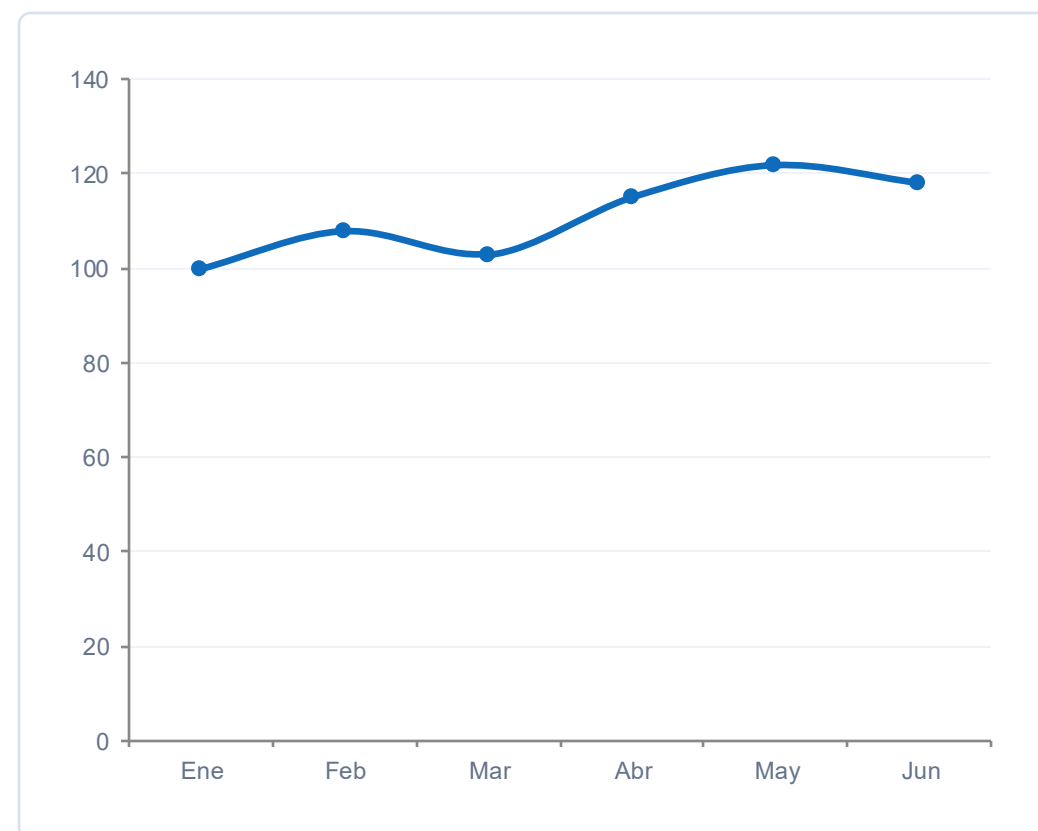
- **Rotación**
 - Bajas sobre plantilla media
- **Headcount**
 - Evolución de la plantilla
- **Equidad salarial**
 - Medianas por grupo
- **Antigüedad**
 - Cohortes y retención

Finanzas

Calcula rendimientos, volatilidad y medias móviles sobre series financieras con funciones de ventana.

In [1]:

```
precios["retorno"] = precios["cierre"].pct_change()
precios["vol_30d"] = precios["retorno"].rolling(30).std()
precios["ma_50"] = precios["cierre"].rolling(50).mean()
precios["ma_200"] = precios["cierre"].rolling(200).mean()
```



Marketing

Mide adquisición, conversión y retención. El análisis de cohortes revela cómo evoluciona cada grupo de clientes.

In [1]:

```
# Tasa de conversión por canal
df.groupby("canal").agg(
    visitas=("visita", "sum"),
    conversiones=("compra", "sum"),
    tasa=("compra", "mean"),
)
# Cohorte por mes de alta
df.groupby([df["alta"].dt.to_period("M"), "mes_actividad"]).size()
```

Analítica de marketing

- **Conversión**
 - Compras sobre visitas por canal
- **Cohortes**
 - Retención por mes de alta
- **CAC / LTV**
 - Coste y valor del cliente
- **Atribución**
 - Aporte de cada canal

IoT

Los sensores generan series de alta frecuencia. Resample las agrega a intervalos regulares y manejables.

In [1]:

```
sensores = sensores.set_index("timestamp")

# De lecturas por segundo a medias por minuto
por_min = sensores["temp"].resample("1min").mean()

# Detección simple de anomalías (3 sigma)
z = (por_min - por_min.mean()) / por_min.std()
anomalias = por_min[z.abs() > 3]
```

Patrones IoT

- **resample**
- Reagrupa por intervalos de tiempo
- **Interpolación**
- Rellena lecturas ausentes
- **Ventanas móviles**
- Suaviza el ruido del sensor
- **Anomalías**
- Umbrales o z-score

Machine Learning con Scikit-Learn

Pandas prepara los features; Scikit-learn entrena. Su interoperabilidad hace fluido el flujo de modelado.

In [1]:

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

X = pd.get_dummies(df[features])      # one-hot desde Pandas
y = df["objetivo"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2)
modelo = RandomForestClassifier().fit(X_tr, y_tr)
```

Pandas → ML

- **get_dummies**
- Codificación one-hot de categóricas
- **DataFrame → sklearn**
- Acepta DataFrames directamente
- **train_test_split**
- Partición reproducible
- **Feature engineering**
- El paso donde Pandas brilla

MÓDULO 10

Nivel experto

MultilIndex, series temporales, feature engineering e integraciones con Spark, Azure y Power BI.

Multindex

Un índice jerárquico permite modelar múltiples dimensiones (región × año) sobre una tabla plana.

In [1]:

```
df = df.set_index(["region", "anio"]).sort_index()

df.loc[("Norte", 2026)]           # selección por nivel
df.xs(2026, level="anio")         # corte transversal
df.groupby(level="region").sum()   # agrega por un nivel
```

Herramientas

- **set_index([...])**
- Crea el índice jerárquico
- **.xs(level=...)**
- Corte por un nivel concreto
- **groupby(level=...)**
- Agrega por nivel
- **unstack / stack**
- Pivotar entre índice y columnas

Time Series

Con un DatetimeIndex, Pandas ofrece indexado por fechas, selección por periodos y aritmética temporal.

In [1]:

```
df = df.set_index("fecha")
df["2026-06"]           # todo junio de 2026
df["2026-01":"2026-03"] # primer trimestre
df.shift(1)             # desplaza una posición
df.asfreq("D").ffill()  # frecuencia diaria
```

Superpoderes temporales

- **Slicing por fecha**
- Selecciona periodos con strings
- **shift / diff**
- Comparaciones periodo a periodo
- **Zonas horarias**
- tz_localize y tz_convert
- **Frecuencias**
- asfreq alinea a un calendario

Resampling

resample cambia la frecuencia temporal: downsampling para agregar o upsampling para densificar.

In [1]:

```
df["ventas"].resample("M").sum()      # a mensual (agrega)
df["ventas"].resample("W").mean()     # a semanal

df.resample("D").ffill()              # a diario (rellena)
df.resample("Q").agg(["sum", "mean"]) # varias métricas
```

Dos direcciones

- **Downsampling**
 - De fino a grueso: agrega (sum, mean)
- **Upsampling**
 - De grueso a fino: rellena o interpola
- **Reglas de frecuencia**
 - D, W, M, Q, Y y personalizadas

Feature Engineering

Crear buenas variables suele impactar más en un modelo que el propio algoritmo. Pandas es la herramienta ideal.

In [1]:

```
df["ratio"] = df["deuda"] / df["ingreso"]
df["mes"] = df["fecha"].dt.month
df["lag_1"] = df.groupby("cliente")["gasto"].shift(1)
df["media_movil"] = (df.groupby("cliente")["gasto"]
                    .transform(lambda s: s.rolling(3).mean()))
df = pd.get_dummies(df, columns=["segmento"])
```

Tipos de features

- **Ratios y combinaciones**
 - Relaciones entre columnas
- **Temporales**
 - Componentes de fecha y lags
- **Agregados por grupo**
 - transform mantiene la forma
- **Codificación**
 - one-hot y ordinal

Integración con NumPy

Pandas se construye sobre NumPy: puedes bajar al array subyacente para cálculos vectorizados de alto rendimiento.

In [1]:

```
import numpy as np

arr = df[["x", "y"]].to_numpy()    # DataFrame -> ndarray
df["z"] = np.log1p(df["ventas"])  # ufuncs vectorizadas
df["cat"] = np.where(df["s"] > 0, "alta", "baja")
df["norm"] = (df["v"] - np.mean(df["v"])) / np.std(df["v"])
```

Puente NumPy

- **to_numpy()**
- Accede al array subyacente
- **ufuncs**
- np.log, np.exp, np.where vectorizadas
- **Rendimiento**
- Cálculo numérico a bajo nivel
- **Compatibilidad**
- NumPy 2.0 desde Pandas 2.2

Integración con Spark

Cuando los datos superan una máquina, la API de Pandas sobre Spark permite escalar sin reaprender la sintaxis.

In [1]:

```
# API de Pandas sobre Spark (distribuido)
import pyspark.pandas as ps
psdf = ps.read_parquet("abfss://data/ventas")
resumen = psdf.groupby("region")["importe"].sum()

# Conversión puntual Spark -> Pandas
pdf = spark_df.toPandas()
```

Estrategias de escala

- **pyspark.pandas**
- Misma sintaxis, ejecución distribuida
- **toPandas()**
- Trae un subconjunto a memoria
- **Cuándo escalar**
- Cuando no cabe en una sola máquina
- **Notebooks Fabric**
- Spark y Pandas conviven

Integración con Azure Data Lake

Pandas lee y escribe directamente en un data lake usando rutas ABFS y credenciales gestionadas.

In [1]:

```
storage = {"account_name": "milake",
           "account_key": "<clave>"}

df = pd.read_parquet(
    "abfs://contenedor/ventas/2026.parquet",
    storage_options=storage)

df.to_parquet("abfs://contenedor/salida/limpio.parquet",
              storage_options=storage)
```

Claves de acceso

- **Rutas abfs://**
- Acceso directo a OneLake / ADLS
- **storage_options**
- Credenciales y configuración
- **Parquet + Delta**
- Formatos nativos del lakehouse
- **Sin descargar**
- Lee remoto de forma perezosa

Integración con Fabric y Power BI

En Microsoft Fabric, Pandas es la librería de referencia para explorar datos del lakehouse y conectarlos con Power BI.

- **Notebooks de Fabric**
 - Pandas explora datos de OneLake al instante
- **Spark ↔ Pandas**
 - toPandas() para el análisis fino
- **Semantic Link (SemPy)**
 - Conecta modelos de Power BI a DataFrames
- **Ejecuta DAX desde Python**
 - Consulta medidas del modelo semántico
- **Del lakehouse al informe**
 - Pandas prepara; Power BI visualiza
- **SDK de Semantic Link**
 - pip install semantic-link

MÓDULO

Cierre y recursos

Errores comunes, buenas prácticas, cheat sheet, comparativas y tu roadmap a experto.

Errores más comunes

Evita estas trampas habituales. Cada tarjeta muestra el patrón incorrecto y su alternativa correcta.

Indexado encadenado

✗ `df[df.a>0]['b'] = 1`

✓ `df.loc[df.a>0, 'b'] = 1`

Iterar con iterrows

✗ `for i,r in df.iterrows(): ...`

✓ `df['c'] = df['a'] * df['b']`

Ignorar dtypes

✗ `pd.read_csv('f.csv')`

✓ `read_csv(..., dtype=..., parse_dates=...)`

Abusar de inplace

✗ `df.drop(col, inplace=True)`

✓ `df = df.drop(columns=col)`

Buenas prácticas profesionales

Hábitos que distinguen a un usuario avanzado de Pandas y hacen tu código fiable y mantenible.

- **Vectoriza siempre**
- Evita bucles; usa operaciones de columna
- **Encadena métodos**
- Pipelines legibles y sin variables sueltas
- **Tipa y valida pronto**
- dtypes y comprobación de esquema al cargar
- **Usa .loc para asignar**
- Evita el SettingWithCopyWarning
- **Perfila antes de optimizar**
- Mide con %timeit y memory_usage
- **Documenta y prueba**
- Funciones puras y tests sobre los datos

Cheat Sheet de Pandas

Referencia rápida de las operaciones más usadas, organizadas por categoría.

Inspección

```
df.head()
df.info()
df.describe()
df.shape
df.dtypes
df.isna().sum()
```

Selección

```
df['col']
df.loc[m, cols]
df.iloc[0:5]
df.query('a>0')
df[df.a > 0]
df.filter(like=)
```

Transformación

```
df.groupby('k').agg()
df.merge(o, on=)
df.pivot_table()
df.sort_values()
df.assign()
df.apply()
```

Limpieza

```
df.dropna()
df.fillna(0)
df.drop_duplicates()
df.astype()
pd.to_datetime()
df.rename()
```

Comparativa con Polars

Polars es una alternativa en Rust, multihilo y con evaluación perezosa, pensada para grandes volúmenes.

Pandas

- Ecosistema maduro y omnipresente
- Ejecución mayormente en un solo hilo
- Evaluación inmediata (eager)
- Integración total con ML y viz
- El estándar para aprender y prototipar

Polars

- Núcleo en Rust, multihilo por defecto
- API perezosa: optimiza el plan de consulta
- Formato Arrow y modo streaming
- ~5–6× más rápido en cargas pesadas*
- Ecosistema aún en maduración

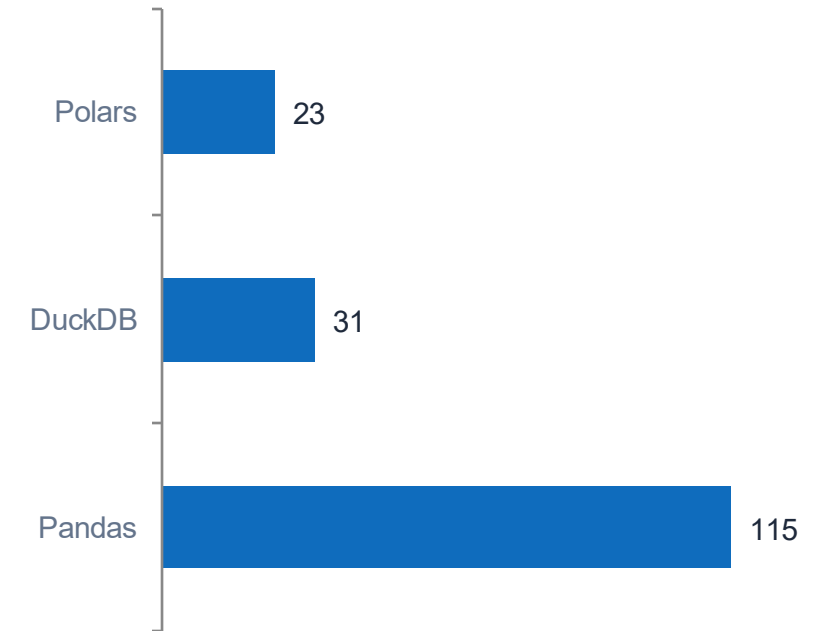
Comparativa con DuckDB

DuckDB es un motor SQL analítico (OLAP) embebido: el «SQLite de la analítica». Consulta archivos en disco sin cargarlos en memoria.

- **SQL embebido**
- Motor OLAP en proceso, sin servidor
- **Consulta Parquet/CSV directo**
- Sin ETL previo ni cargar todo en RAM
- **Vuelca a disco**
- Procesa datos mayores que la memoria
- **Interopera con Arrow**
- Convive con Pandas y Polars

**Cifras ilustrativas de un benchmark único; varían según hardware y datos.*

Group-by · 100 M filas (seg)*



Recursos para aprendizaje avanzado

Sigue creciendo con estas fuentes de referencia y práctica deliberada.

- **Documentación oficial**
 - pandas.pydata.org: guía de usuario y API
 - **«Python for Data Analysis»**
 - El libro de Wes McKinney, creador de Pandas
 - **Practica con datos reales**
 - Kaggle, datos abiertos y proyectos propios
- **Explora el ecosistema**
 - Polars, DuckDB, Arrow y Scikit-learn
- **Lee las release notes**
 - Novedades de cada versión (2.x → 3.x)
- **Comunidad**
 - Stack Overflow, GitHub y blogs técnicos

Roadmap: conviértete en experto en Pandas

- 1 Principiante**
Series, DataFrame, selección y filtrado básico.
- 2 Intermedio**
Limpieza, groupby, merge, pivot y visualización.
- 3 Avanzado**
Rendimiento, MultiIndex, series temporales y EDA.
- 4 Experto**
Feature engineering e integración con Spark, Azure y Fabric.



Conclusiones

Lo esencial que llevarte de este curso.

- **Pandas es el estándar de datos en Python**
- Puente entre fuentes, análisis, viz y ML
- **Domina los fundamentos**
- Series, DataFrame, índices y tipos lo sostienen todo
- **Piensa en vectorizado**
- Más velocidad, memoria y código limpio
- **La calidad de datos es primero**
- Limpieza y validación antes de modelar
- **Escala cuando toque**
- Polars, DuckDB o Spark según el volumen
- **El aprendizaje es continuo**
- Practica con datos reales y sigue el roadmap

¡Gracias!

Preguntas y respuestas

Abramos el turno de dudas: filtrado, rendimiento, integraciones o tu caso de uso concreto.

